

VCGen

Boogie Style

Idea

Boogie-style VCGen uses backward safety equations over the CFG.

For each block i , introduce:

$$ok_i$$

ok_i means: starting execution at block i is safe.

The counterexample query is:

$$\neg ok_{\text{entry}}$$

No path selection, no block-reachability variables — control flow becomes recursion.

Block Equation

$$\text{ok}_i \Leftrightarrow \left(\text{defs}_i \Rightarrow \left(\text{asserts}_i \wedge \bigwedge_{j \in \text{succ}(i)} \left((\text{cond}_{i,j} \wedge \text{dsa}_{i,j}) \Rightarrow \text{ok}_j \right) \right) \right)$$

- assumptions and definitions are premises
- assertions are consequents
- each feasible successor must be safe
- infeasible blocks are safe vacuously

Terminal Block

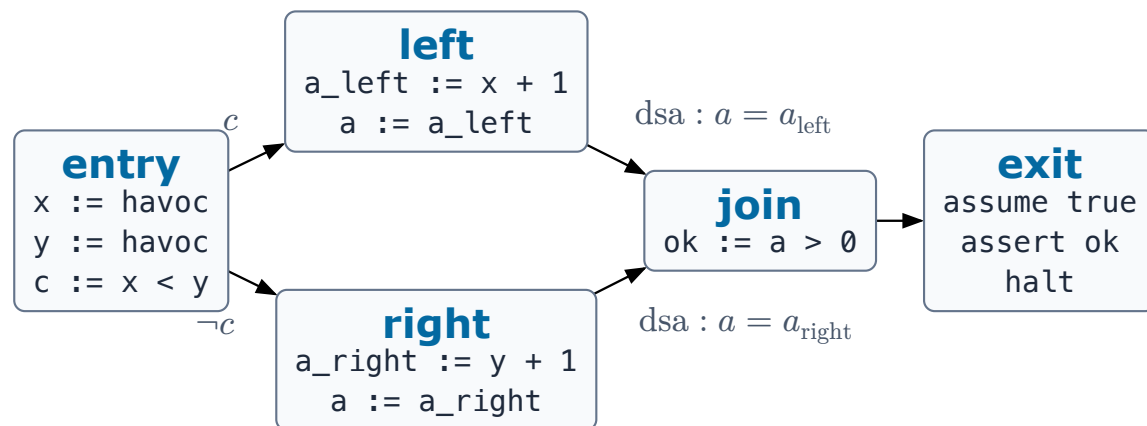
For a terminal block, the successor conjunction is true:

$$ok_i \Leftrightarrow (defs_i \Rightarrow asserts_i)$$

A false reachable assertion makes the local `ok_i` false, then unsafety propagates backward to entry.

The Diamond, In DSA Form

Boogie-style consumes DSA: the merge moves onto the incoming edges.



The assignments to `a` are **edge-sensitive DSA definitions** — they enter the formula on the edge premises, not as block facts.

Encoded Equations

$$\mathbf{OK}_{\text{entry}} : \text{ok}_{\text{entry}} \Leftrightarrow (c = (x < y) \Rightarrow ((c \Rightarrow \text{ok}_{\text{left}}) \wedge (\neg c \Rightarrow \text{ok}_{\text{right}})))$$

$$\mathbf{OK}_{\text{left}} : \text{ok}_{\text{left}} \Leftrightarrow (a_{\text{left}} = x + 1 \Rightarrow ((a = a_{\text{left}}) \Rightarrow \text{ok}_{\text{join}}))$$

$$\mathbf{OK}_{\text{right}} : \text{ok}_{\text{right}} \Leftrightarrow (a_{\text{right}} = y + 1 \Rightarrow ((a = a_{\text{right}}) \Rightarrow \text{ok}_{\text{join}}))$$

Encoded Exit

$$\mathbf{OK}_{\text{join}} : \text{ok}_{\text{join}} \Leftrightarrow (\text{ok} = (a > 0) \Rightarrow \text{ok}_{\text{exit}})$$

$$\mathbf{OK}_{\text{exit}} : \text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$$

$$\mathbf{DISCHARGE} : \neg \text{ok}_{\text{entry}}$$

The model chooses havoc values and a branch whose successor is unsafe.

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2$, $y = 0$, $a = a_{\text{left}} = -1$, $\text{ok} = \text{false}$

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2$, $y = 0$, $a = a_{\text{left}} = -1$, $\text{ok} = \text{false}$
- OK_{exit} : $\text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$, so $\text{ok}_{\text{exit}} = \text{false}$

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2$, $y = 0$, $a = a_{\text{left}} = -1$, $\text{ok} = \text{false}$
- OK_{exit} : $\text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$, so $\text{ok}_{\text{exit}} = \text{false}$
- OK_{join} : premise $\text{ok} = (a > 0)$ holds ($-1 > 0$ is false), successor unsafe $\rightarrow \text{ok}_{\text{join}} = \text{false}$

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2$, $y = 0$, $a = a_{\text{left}} = -1$, $\text{ok} = \text{false}$
- OK_{exit} : $\text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$, so $\text{ok}_{\text{exit}} = \text{false}$
- OK_{join} : premise $\text{ok} = (a > 0)$ holds ($-1 > 0$ is false), successor unsafe $\rightarrow \text{ok}_{\text{join}} = \text{false}$
- OK_{left} : premises $a_{\text{left}} = x + 1 = -1$ and edge definition $a = a_{\text{left}}$ hold $\rightarrow \text{ok}_{\text{left}} = \text{false}$

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2$, $y = 0$, $a = a_{\text{left}} = -1$, $\text{ok} = \text{false}$
- OK_{exit} : $\text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$, so $\text{ok}_{\text{exit}} = \text{false}$
- OK_{join} : premise $\text{ok} = (a > 0)$ holds ($-1 > 0$ is false), successor unsafe $\rightarrow \text{ok}_{\text{join}} = \text{false}$
- OK_{left} : premises $a_{\text{left}} = x + 1 = -1$ and edge definition $a = a_{\text{left}}$ hold $\rightarrow \text{ok}_{\text{left}} = \text{false}$
- OK_{entry} : $c = (x < y) = \text{true}$, and the true branch requires $c \Rightarrow \text{ok}_{\text{left}} \rightarrow \text{ok}_{\text{entry}} = \text{false}$

A Model, Walked Backward

The diamond is unsafe (a can be ≤ 0). Watch the solver's model flow through the equations:

- model picks $x = -2$, $y = 0$, $a = a_{\text{left}} = -1$, $\text{ok} = \text{false}$
- OK_{exit} : $\text{ok}_{\text{exit}} \Leftrightarrow \text{ok}$, so $\text{ok}_{\text{exit}} = \text{false}$
- OK_{join} : premise $\text{ok} = (a > 0)$ holds ($-1 > 0$ is false), successor unsafe $\rightarrow \text{ok}_{\text{join}} = \text{false}$
- OK_{left} : premises $a_{\text{left}} = x + 1 = -1$ and edge definition $a = a_{\text{left}}$ hold $\rightarrow \text{ok}_{\text{left}} = \text{false}$
- OK_{entry} : $c = (x < y) = \text{true}$, and the true branch requires $c \Rightarrow \text{ok}_{\text{left}} \rightarrow \text{ok}_{\text{entry}} = \text{false}$
- **DISCHARGE** $\neg \text{ok}_{\text{entry}}$ is satisfied — the model **is** the failing execution entry $\rightarrow$ left $\rightarrow$ join $\rightarrow$ exit.

Static ITE Merge Variant

The merge can be compiled into an ordinary definition:

```
join:  
  a := ite(c, a_left, a_right)  
  ok := a > 0  
  goto exit
```

$$\text{ok}_{\text{join}} \Leftrightarrow ((a = \text{ite}(c, a_{\text{left}}, a_{\text{right}}) \wedge \text{ok} = (a > 0)) \Rightarrow \text{ok}_{\text{exit}})$$

Now `left` and `right` have no edge definitions — the predecessor-sensitive merge became a local fact of `join`.

Tradeoffs

Advantages:

- no separate path-selection formula
- many assertions are natural
- assumptions are handled by implication

Costs:

- formula is nested
- cross-block simplification is harder
- ϕ /ITE extensions need exclusivity side conditions

Phi Extension Needs CFG Facts

If a phi is compiled as:

$$a = \text{ite}(\text{bb}_{\text{left}}, a_{\text{left}}, a_{\text{right}})$$

then the block variables used by the ITE must be consistent:

$$\begin{aligned} \text{bb}_{\text{join}} &\Rightarrow \text{bb}_{\text{left}} \vee \text{bb}_{\text{right}} \\ &\neg(\text{bb}_{\text{left}} \wedge \text{bb}_{\text{right}}) \end{aligned}$$

The backward equations prove safety; the CFG facts keep the ITE merge faithful — the same exclusivity pitfall as SeaHorn's phi-as-ITE.

Same Diamond, Two Formulas

SeaHorn — flat conjunction:

$$\begin{aligned} & \text{bb}_{\text{entry}} \wedge \text{bb}_{\text{exit}} \wedge \dots \\ & \text{bb}_{\text{entry}} \Rightarrow c = (x < y) \\ & \text{bb}_{\text{left}} \Rightarrow a_{\text{left}} = x + 1 \\ & (\text{bb}_{\text{entry}} \wedge \text{bb}_{\text{left}}) \Rightarrow c \\ & (\text{bb}_{\text{left}} \wedge \text{bb}_{\text{join}}) \Rightarrow a = a_{\text{left}} \\ & \text{bb}_{\text{exit}} \Rightarrow \neg \text{ok} \end{aligned}$$

facts at top level; path selected by block variables

Boogie — nested recursion:

$$\begin{aligned} & \neg \text{ok}_{\text{entry}} \\ & \text{ok}_{\text{entry}} \Leftrightarrow (c = (x < y) \Rightarrow \\ & ((c \Rightarrow \text{ok}_{\text{left}}) \wedge (\neg c \Rightarrow \text{ok}_{\text{right}}))) \\ & \text{ok}_{\text{left}} \Leftrightarrow (a_{\text{left}} = x + 1 \Rightarrow \\ & ((a = a_{\text{left}}) \Rightarrow \text{ok}_{\text{join}})) \\ & \dots \end{aligned}$$

facts nested under equations; path selected by implication chains

Same models, same verdict — different levers for the solver: SeaHorn exposes equalities for substitution; Boogie gets assertions and assumptions for free.